

Lab Week 7 - Unit & E2E Testing

- Due No Due Date
- Points 3

Lab 7 Logistics

Due Tuesday, May 19th, at 11:59pm.

Working in teams

To encourage team bonding and to get to know people on your team better, you can work on this lab with a group member from your existing project team. If your existing team has an odd number of teammates, you can only work on a team of 3 if all other teammates have paired up. ***You cannot work with the same partner as last time nor can you work with someone on another team.***

In this lab, we are going to be walking you through End-to-End (E2E) testing. This style of testing tries to replicate a user's workflow from start to finish (i.e. from end-to-end). As such, it will be more focused on interacting with the webpage.

FAQ

[FAQ - Lab Week 7](#)

<https://docs.google.com/document/d/1lcUr5yWcs4d2MnWnkkZBQc7YdZm7E4SgsUzJlqcXsok/edit?usp=sharing>

Resources

- <https://jestjs.io/docs/en/getting-started>  <https://jestjs.io/docs/en/getting-started>
- <https://pptr.dev/>  <https://pptr.dev/>
- <https://pptr.dev/#?product=Puppeteer&version=v11.0.0&show=api-pageselector-1>  <https://pptr.dev/#?product=Puppeteer&version=v11.0.0&show=api-pageselector-1>
- <https://github.com/smooth-code/jest-puppeteer>  <https://github.com/smooth-code/jest-puppeteer>

Set Up

- Join the #lab-7 channel on Slack to ask or help answer your classmates' questions
- Click the 'use this template' button in the upper right to get the starter code from this repo: https://github.com/CSE110-SP26/Lab7_Starter  (https://github.com/CSE110-SP26/Lab7_Starter) and add your lab partner as a collaborator if needed.
- Run this terminal command:

```
npm -v
```

- (checks if you have the Node Package Manager aka npm installed -- this is a tool that allows you to access and download resources such as external library code written by other people in the form of software packages that npm locates within a giant registry)
 - If you don't have it on your local machine, install it here  (<https://www.npmjs.com/get-npm>) <https://www.npmjs.com/get-npm>  (<https://www.npmjs.com/get-npm>)
- run this command:

```
npm install --save-dev jest babel-jest @babel/core @babel/preset-env puppeteer jest-puppeteer
```

- This installs Jest, the Javascript testing framework we'll be using, and Babel, a Javascript compiler that allows us to use ES6 modules over CommonJS.
- Note: "--save-dev" includes these packages only during development. Since puppeteer and jest are just for testing, there's no need to include the extra dependencies when you deploy for projects for production
- Add the new *node_modules* directory to your **.gitignore** inside your top-level directory to prevent code bloat committed to your repository.
- If the installation does not automatically produce a **package.json** file for you, you can create a blank one and fill it out with the data in the section below

Configuring Jest & Puppeteer

- First, configure your npm script to run Jest. In your package.json, add a *scripts* section with the following:

```
"scripts": {  
  "test": "jest"  
}
```

This will tell your node interpreter to run the "jest" command when we run "npm run test".

- Configure your node project
 - Inside of your **package.json** file, inside the JSON object at the top level (e.g. right before the last curly brace `}`), add the following in order to run the “jest” command with the “jest-puppeteer” framework, with verbose output

```
"jest": {  
  "preset": "jest-puppeteer",  
  "verbose": true  
}
```

your package.json file should end up like this:

```
{  
  "devDependencies": {  
    "@babel/core": "^7.24.5",  
    "@babel/preset-env": "^7.24.5",  
    "babel-jest": "^29.7.0",  
    "jest": "^29.7.0",  
    "jest-puppeteer": "^10.0.0",  
    "puppeteer": "^22.7.1"  
  },  
  "scripts": {  
    "test": "jest"  
  },  
  "jest": {  
    "preset": "jest-puppeteer",  
    "verbose": true  
  }  
}
```

Intro

Software testing has become an absolutely integral part of the software development pipeline. Every software company integrates many different forms of software testing procedures and frameworks in order to ensure the quality of the software they are producing.

The most recent developments in the practice of testing is the increased use of automation testing, in which scripts are written to automatically test your new and existing software based on certain events, i.e. code getting pushed to the remote repository, or test runs of the whole repo that are triggered nightly.

In this lab, we are going to be walking you through one of the ways to test the front-end of your applications, namely **end-to-end tests**. We will use **Puppeteer** to operate the browser and **Jest** to perform the tests.

Check Your Understanding

Fill your answer in your **README.md**

1) Where would you fit your automated tests in your Recipe project development pipeline? Select one of the following and explain why.

1. Within a Github action that runs whenever code is pushed
2. Manually run them locally before pushing code
3. Run them all after all development is completed

Expose: E2E Testing with Jest-Puppeteer

The style of testing we'll be looking at is **end-to-end testing**, also named **UI Testing** or **E2E testing**. End-to-end testing is a way for developers to automate test cases that involve emulating user actions from start to finish (end to end). We will be using an api called Puppeteer which provides developers with tools that enable you to interact with a web page by navigating to its url and interacting with its DOM.

Check Your Understanding

2) Would you use an end to end test to check if a function is returning the correct output? (yes/no)

In this lab you will be testing a mock store webapp that works very similarly to your recipe card website from Lab 6. The URL of the store webapp can be found at <https://cse110-sp25.github.io/CSE110-Shop/>  (<https://cse110-sp25.github.io/CSE110-Shop/>). The code for the webapp can also be found in the src folder of the starter code.

Open the `__tests__/lab7.test.js` file

This is where you will write your Puppeteer tests. The `describe()` function creates a test suite for your SPA app. A test suite is essentially a collection of tests that all execute to test the same area of code. Defining test suites allows you to encapsulate all of the test logic so that tests from one suite don't affect the results of tests in another suite.

We've included the first two tests for you that are mostly complete so that you can generally see how they operate. You first interact with the webpage using **Puppeteer** (that's the **Page** variable you're seeing, along with some other things), then you use **Jest** to create

tests (e.g. **expect(something).toBe(something)** - this is but one example, **Jest** has a whole suite of things you can use to test with).

The web app also uses web components to display product items, and `localStorage` to store them. We went more in depth with those in Lab 6. For Lab 7, you can treat each component as having its own mini DOM tree (which **shadowRoot** is the root of) and **localStorage** as a built in browser (NOT puppeteer) variable that you can fetch specific data from using **.getItem()**. To view your `localStorage`, in the browser go to devTools -> applications -> `localStorage` -> your domain name.

Since Puppeteer can be a bit overwhelming at first, here are some concepts to get you started (you should read the documentation to learn more about these in depth):

- Puppeteer **handles**: essentially a puppeteer object that points to the actual DOM object, and is how puppeteer manipulates page elements. When trying to query a page element, check to see if the method you are using returns the type you are expecting.
- `page.$(selector)`: has to be called either on page or on a handle, and returns a handle. It calls `.querySelector(selector)` on the document (if page) or the DOM object that the handle is pointing to.
- `.getProperty(property)`: gets the property of the handle it is called on, and returns a handle.
- `.$eval(selector, function)`: has to be called on a handle. Queries an element using the selector, and passes it into a function. Returns serializable data.
- `page.evaluate(function)`: Runs a function in the browser context.

We've outlined 8 steps for you to complete in **lab7.test.js** to complete the testing suite, marked by the TODO comments. Follow the instructions for each to complete each of the tests. All the incomplete tests are initialized with an `it.skip()`. Make sure to remove the `.skip` in order to run the test after you have finished filling out the corresponding TODO. It may be useful for you to consult the code of the website, which can be found in the `src` folder. After finishing the tests, please take a **screenshot** of your terminal showing the results of running the tests.

Some helpful resources:

Puppeteer Docs: <https://pptr.dev/> ↗ [\(https://pptr.dev/\)](https://pptr.dev/)

Puppeteer Page Selector Docs: <https://pptr.dev/#?product=Puppeteer&version=v11.0.0&show=api-pageselector-1> ↗ [\(https://pptr.dev/#?product=Puppeteer&version=v11.0.0&show=api-pageselector-1\)](https://pptr.dev/#?product=Puppeteer&version=v11.0.0&show=api-pageselector-1)

Jest Docs: <https://jestjs.io/docs/getting-started> ↗ [\(https://jestjs.io/docs/getting-started\)](https://jestjs.io/docs/getting-started)

To run your tests, use the command:

```
npm run test
```

To run a specific test file, use the command:

```
npm test path-to-test-file
```

NOTE: An empty test case will default to a pass on a test run.

If you are having an issue with tests opening multiple instances of chrome, add the `-maxWorkers=1` flag to your `package.json`.

```
"scripts": {  
  "test": "jest --maxWorkers=1"  
}
```

Limiting the `maxWorkers` to 1 resolves an issue with `jest-puppeteer` where it will spawn multiple browser instances when in `headless=false` mode. You can remove it to make your tests run faster if it isn't an issue on your machine.

Setting up the webapp locally (OPTIONAL: only if needed)

This is for if you are having any issues connecting to our GitHub pages website. We have included the entire shop site for you locally in the starter code, you can run it by

1. First, open **src/index.html** in VS Code, then right click anywhere in the file and select 'Open with Live Server'.
2. Next, update the URL in **lab7.test.js** to your local URL from your Live Server.

Explore: Lighthouse testing

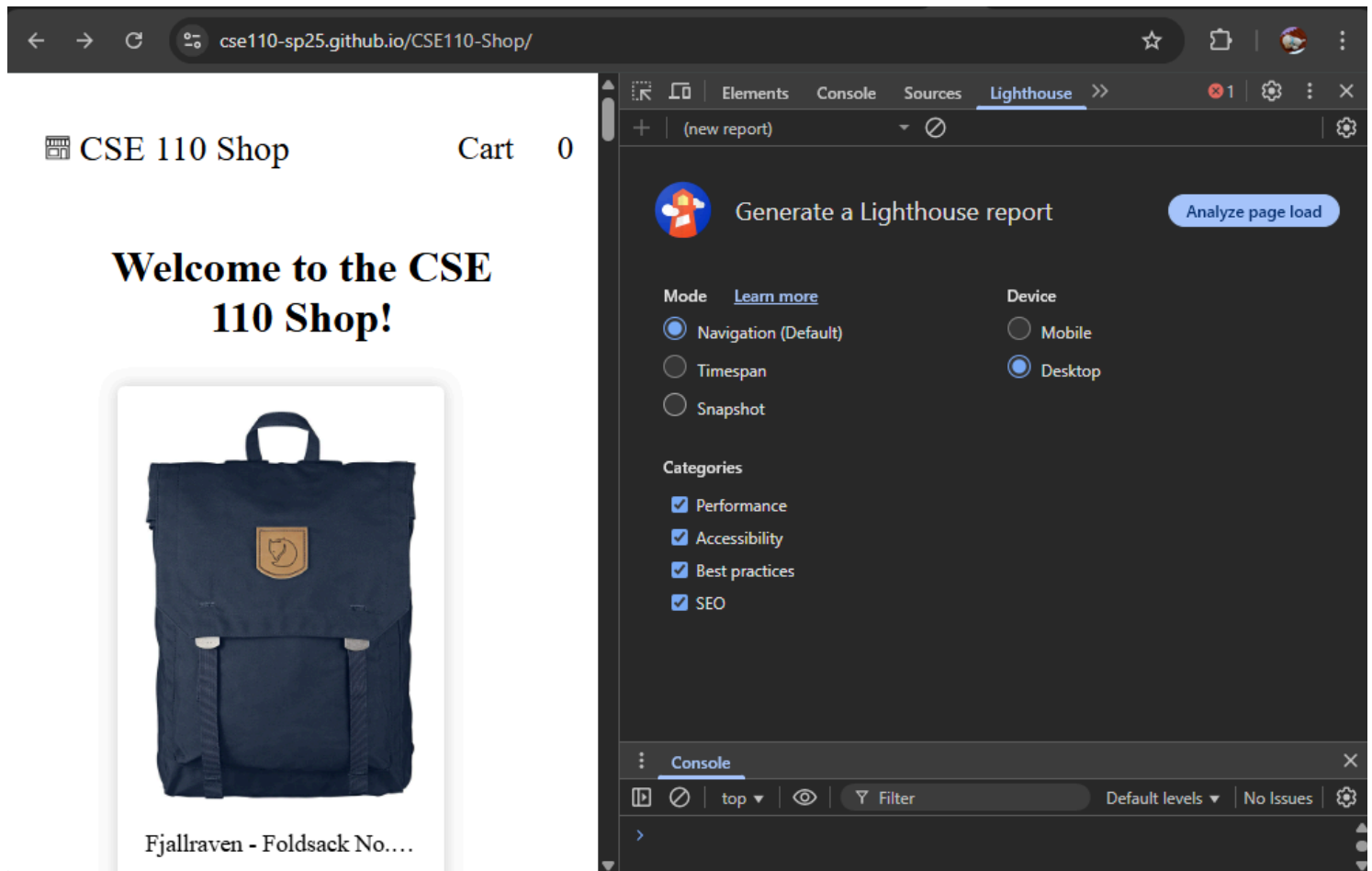
Lighthouse is a tool developed by Google that audits a website's performance and user experience. The easiest way to do use it is with Google Chrome's built in Lighthouse feature. More details and documentation can be found [here](https://github.com/GoogleChrome/lighthouse)  [_\(https://github.com/GoogleChrome/lighthouse\)_](https://github.com/GoogleChrome/lighthouse).

Note: if you don't have Chrome installed on your computer, you are also welcome to do this next part in [PageSpeed Insights](https://pagespeed.web.dev/)  [_\(https://pagespeed.web.dev/\)_](https://pagespeed.web.dev/) instead.

Open the link for CSE 110 shop in Google Chrome, and open DevTools. You should see a "Lighthouse" tab. Navigate there, and make sure that 'Mode' is set to 'Navigation,' 'Device' is set to 'Desktop,' and all the categories are checked.

You can read more about the different Lighthouse modes by going to the documentation, but to sum it up:

- Navigation mode: Analyzes a page right after it loads. Will provide an overall performance metric, but can't analyze interactions or changes in content.
- Timespan mode: Allows the user to record the webpage and then audits the webpage over the recorded span of time. Can be used to get feedback on the performance of user interactions.
- Snapshot mode: Analyzes a page in its current state. Best used for finding accessibility issues. Can't analyze JS performance or changes to the DOM tree.



Click "**Analyze page load.**" After a slight delay, Lighthouse should give you a report that rates your website based on the categories selected. Feel free to try different modes to see how the website performs.

Check Your Understanding

- 3) What is the difference between navigation and snapshot mode?
- 4) Name three things we could do to improve the CSE 110 shop site based on the Lighthouse results.

Expand (No points)

If you would like more practice writing end-to-end tests with puppeteer, we have an additional web app for testing that can be found [here](https://github.com/elaine-ch/CSE110-SP24-Lab6-Template)  (<https://github.com/elaine-ch/CSE110-SP24-Lab6-Template>). Instructions and details are in the README of the repo.

Some extra tasks you could accomplish that could help you out with your project:

- Create a smaller build pipeline for this lab that will automatically run these tests when you commit
- Use Percy <https://percy.io/>  (<https://percy.io/>) - This will let you pix diff different versions of your website, let you know if anything changes / breaks
- Set up some form of accessibility testing in your smaller build pipeline for this lab

What to Submit

- Link to your repo
 - **README.md** with your and your partner's name(s) *and the "Check Your Understanding" question answers*
 - A **screenshot** of all your test results from running `npm test` on **lab7.test.js** in Expose.
 - All of your code
- Make sure your repo is public before you submit!
- If you are working in a group, only **one person** should submit on Gradescope, and then **add the rest of the group to the submission**.